



Module

Système d'Exploitation et Programmation Système

Chapitre 4

Processus & Threads – Programmation (2/2)

Pr. MANALE Boughanja

m.boughanja@ump.ac.ma

Création d'un Processus et Threads

Rappel:

Processus est programme en cours d'exécution, disposant de sa propre mémoire et de ses ressources.

Rappel:

Thread: Sous-ensemble d'un processus, partage la mémoire du processus parent.

Critère	Processus	Thread
Mémoire	Indépendante	Partagée
Communication	IPC	Accès direct aux variables partagées
Création	Fork()	pthread_create() en C threading.Thread() en Python
Valeur retour	Via <code>-1</code> : le processus n'est pas forké ; <code>0</code> : Processus fils qui s'exécute ; <code>>0</code> : C'est le processus père , la valeur retournée est le PID du fils créé.	Via pthread_exit/pthread_join
Type valeur	Int	Void *

Exécution d'un code différent

- L'appel à **fork** provoque la **duplication** du code du processus père pour la création du processus fils.
- On souhaite créer un processus fils disposant d'un code différent de celui du processus père.

N.B:

- ✓ Sous Unix le processus **init** est le seul qui n'a pas de parent
- ✓ Sous windows la création des processus est effectuée par l'appel à la méthode: **create_process()**.

Famille de fonction exec()

Par défaut, l'enfant hérite **du code et des ressources** du parent, comme un enfant hérite des traits génétiques de ses parents. Si on souhaite que l'enfant ait **un comportement différent** (exécute un code différent), il faut lui **attribuer un rôle ou un code spécifique** après sa création, via exec().

- **int execl(const char *path, const char *arg, ...);**
Lance un programme en indiquant son chemin complet et une liste d'arguments séparés.
- **int execlp(const char *file, const char *arg, ...);**
Lance un programme en cherchant son exécutable dans le PATH et en passant une liste d'arguments séparés.
- **int execle(const char *path, const char *arg, ..., char *const envp[]);**
Lance un programme avec une liste d'arguments séparés tout en permettant de définir un environnement personnalisé (envp).
- **int execv(const char *path, char *const argv[]);**
Lance un programme en indiquant son chemin complet et un tableau d'arguments (argv).
- **int execvp(const char *file, char *const argv[]);**
Lance un programme en cherchant l'exécutable dans le PATH et en passant un tableau d'arguments (argv).

Ordre d'exécution:

- Les codes du processus père et des processus fils ne sont plus exécutés dans l'ordre d'écriture.
- Un même code peut être exécuté par plusieurs processus.
- L'exécution des processus est entrelacée par le [scheduler](#),
- l'exécution d'un même programme peut donc provoquer des affichages/ résultats différents.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 int main() {
4     fork();
5     printf("Processus PID=%d\n", getpid());
6     return 0;
7 }
```

Output

```
Processus PID=60954
Processus PID=60955
```

Ou

```
Processus PID=60955
Processus PID=60954
```

L'ordonnanceur décide de l'ordre d'exécution des processus, donc le résultat peut changer à chaque exécution

Processus: Synchronisation

6

Attente de processus fils:

3 fonctions sont à disposition pour surveiller le changement d'état de processus fils (terminaison, pause, continuation) :

- `pid_t wait( int* status )`: C'est la forme la plus simple. Le processus père s'arrête (il est **bloqué**) et attend **qu'un de ses processus fils** se termine. Si plusieurs fils se terminent, le système en choisit un arbitrairement. Pour attendre N processus fils, il faut N appels à **wait**
 - `Pid_t waitpid ( pid_t pid, int *status , int options)`: Cette fonction est plus flexible. Le paramètre `pid` permet de **spécifier quel processus fils attendre**. C'est indispensable lorsque l'ordre de terminaison des fils est critique pour la logique de l'application.
- Ces fonctions permettent au père non seulement de synchroniser sa propre terminaison, mais aussi de **récupérer le statut de sortie du fils**, par exemple, le code passé à **exit()**.

Processus: Synchronisation

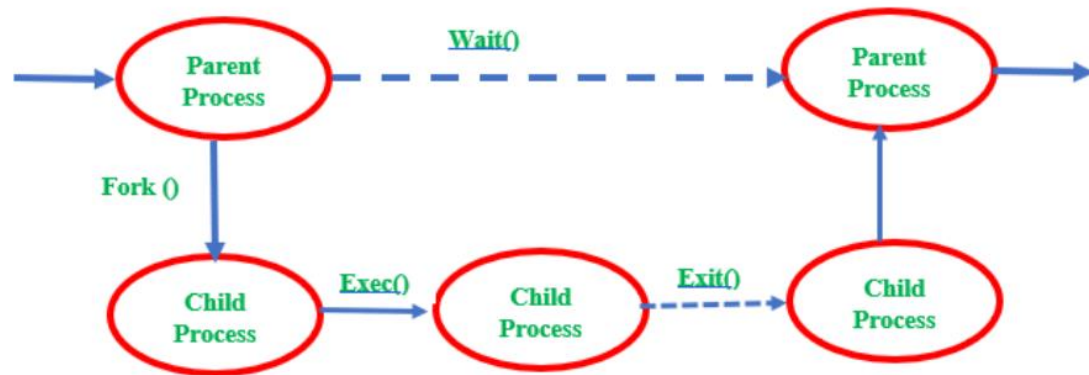
6

Sans appel à `wait` 2 cas possibles si aucune synchronisation entre le processus père et les processus fils :

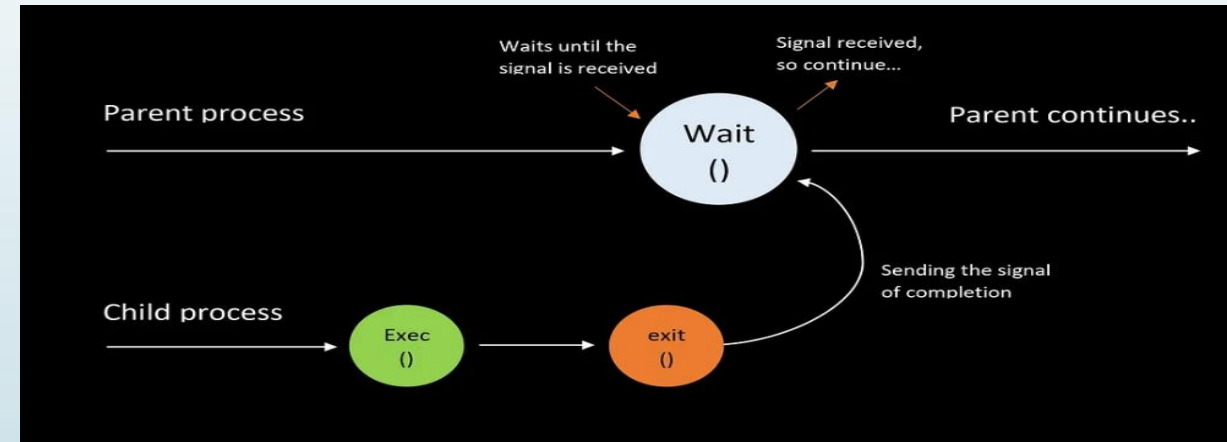
- le processus père se termine avant ses processus fils
- le(s) processus fils se termine(nt) avant le processus père

➔ Orphelin

➔ Zombie



Zombie Process



OrphanProcess

Cas 2: Zombie

8

Si un processus fils n'est pas attendu il reste zombie :

- Jusqu'à la fin du processus père si aucun `wait` du père,
- Ou jusqu'à un appel à `wait` du père récupérant la fin du processus fils.
- La commande `ps aux` : processus zombies identifiés par `Z+`

Remarques sur la synchronisation

10

- L'appel à `sleep` ne doit jamais être utilisé pour synchroniser des processus.
- La synchronisation doit se faire à l'aide de fonctions de synchronisation adéquates comme `wait`.

Processus: Synchronisation

```
1 #include<stdio.h>
2 #include<unistd.h>
3 #include <sys/types.h>
4 #include <sys/wait.h>
5 int main(){
6     char c;
7     pid_t ret=fork();
8     if(ret){
9         printf("Processus %d père du processus %d\n",getpid(), ret);
10        pid_t ret2=fork();
11        if(ret){
12            printf("je suis le père");
13            wait(NULL); wait(NULL);//attente du processus fils
14        }
15        else{
16            printf("je suis le 2 fils");}
17    }
18    else{
19        printf("Processus %d fils du processus %d \n", getpid(),getppid
20            ());
21        scanf("%s", &c ) ;
22    }
23    return 0;
24 }
```

Output



Processus 7428 père du processus 7429
Processus 7429 fils du processus 7428
je suis le père

Remarque:

- Un appel à `wait` récupère la terminaison d'un des processus fils.
- Si plusieurs processus fils sont terminés, le système en choisi un arbitrairement.
- Dans de nombreux cas l'ordre de terminaison importe vous devez utiliser de `pid_t waitpid( pid_t pid, int *status , int options)`

Appel à waitpid

Passage d'un paramètre pid permettant de spécifier le processus fils à attendre :

- < -1 Attente d'un processus fils dont le groupid est -pid (voir setpgid()).
- -1 Attente d'un processus fils (semblable à wait).
- 0 Attente d'un processus fils du même groupe que le processus père.
- > 0 Attente du processus fils pid.

```
1  #include<stdio.h>
2  #include<unistd.h>
3  #include <sys/types.h>
4  #include <sys/wait.h>
5  int main(){
6      char c;
7      pid_t ret=fork();
8      if(ret){
9          printf("Processus %d père du processus %d\n",getpid(), ret);
10         pid_t ret2=fork();
11         if(ret){
12             printf("je suis le père");
13             waitpid(ret,NULL,0); waitpid(ret2,NULL,0); //attente du
                processus fils
14         }
15         else{
16             printf("je suis le 2 fils");}
17     }
18     else{
19         printf("Processus %d fils du processus %d \n", getpid(),getppid
                ());
20         scanf("%s", &c ) ;
21     }
22     return 0;
23 }
```



Output

```
Processus 35266 père du processus 35267
Processus 35267 fils du processus 35266
je suis le père
```

Retour du statut des processus fils

Les fonctions `wait` et `waitpid` permettent au processus père de récupérer le statut d'un processus fils.

- Coté processus fils: `void exit ( int status )`
- Coté processus père: `int status; wait (&status);`

Traitement de la valeur retour status

La valeur status peut être traitée par les fonctions suivantes :

- `WIFEXITED(status)` : vrai si le processus fils s'est terminé normalement
- `WEXITSTATUS(status)` : code retour passé à exit
- D'autres fonctions sont disponibles pour traiter les cas où l'état du processus fils est modifié par des signaux : `man wait`.



Module

Système d'Exploitation et Programmation Système

Chapitre 4

Processus & Threads – Programmation (2/2)

Pr. MANALE Boughanja

m.boughanja@ump.ac.ma